Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

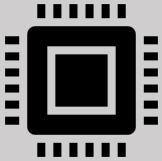
División de Ciencias de la ingeniería

Ingeniería

Manejo e implementación de archivos Sección “A”

Ing. Bryan Misael Monzón Fuentes y Auxiliar Levi Hernández

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual técnico Proyecto #2



Descripción:

En el siguiente documento se da a conocer los detalles a tener en cuenta sobre la presente práctica. Sobre todo si se desea ampliar las funcionalidades de la aplicación se deben de conocer a detalle cómo es que funciona el sistema ya que está hecho todo de forma que sea lo más cómodo de visualizar y de encontrar por cada caso de uso planteado en la arquitectura por capas.

Disclaimer: Todo el código se encuentra escrito en ingles

“Bienvenido programador”

Como primer punto se requiere contar con los pasos para poder ejecutar el código del repositorio en la computadora local del programador, al igual que conocer la distribución de carpetas que se hizo. No sin antes mencionar todas las metodologías, tecnologías y patrones de diseño utilizados.

Tecnologías utilizadas para la realización del proyecto:

- **Nest 11:** Utilizando como lenguaje principal Typescript sobre el que se montó la aplicación haciendo uso del framework de Nest. Esta cuenta con su respectivo código dividido y organizado donde se puede interactuar con el backend.
- **Angular 21:** Es la principal herramienta en la que se creó la aplicación cliente. Donde se encuentra alojado todo lo que se presenta en la web por lo tanto si no se tiene es importante que se tenga estrictamente la versión 22 de node.

Si no tiene Nest hay que instalarlo para poder usar su CLI

https://docs-nestjs-com.translate.goog/cli/overview?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true

Si no tiene node hay que instalarlo y luego instalar angular cli

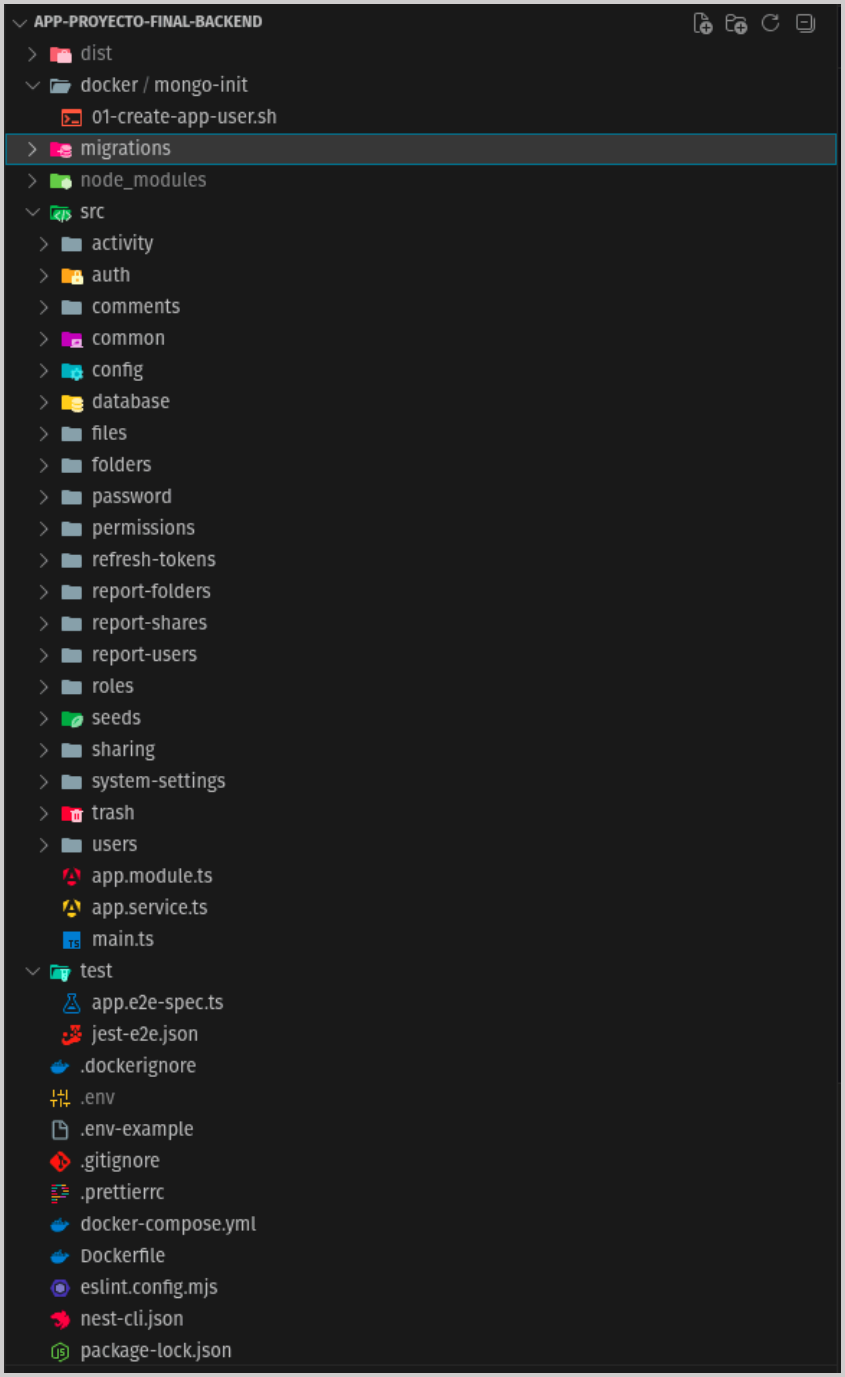
Sitio oficial de node: https://nodejs.org/es/download

Sitio oficial de angular: https://angular.dev/installation

- **Tailwind:** Es el principal motor de estilos de la aplicación cliente.
- **HTML:** Es el principal motor de estructura de las vistas de la aplicación.
- **Typescript:** Es el principal motor de manejo de lógica de la aplicación cliente y servidor.
- **NodeJS:** Principal gestor de dependencias del proyecto de ambas aplicaciones.
- **Github:** Repositorio remoto de la aplicación.

DIRECTORIO DE ARCHIVOS APLICACIÓN BACKEND

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de variables y archivos.



Detalles importantes:

- **La carpeta “common”:** es donde se encuentra toda la lógica en común que maneja la aplicación y configuraciones de seguridad y sobre todo los decoradores principales utilizados en la aplicación.
- **La carpeta “config”:** es donde está alojada toda la lógica de configuración inicial de la aplicación.
- **La carpeta: “seeds”:** Maneja toda la lógica de los scripts y datos iniciales que requiere la aplicación para poder poblar la base de datos y comprobar su correcto funcionamiento.

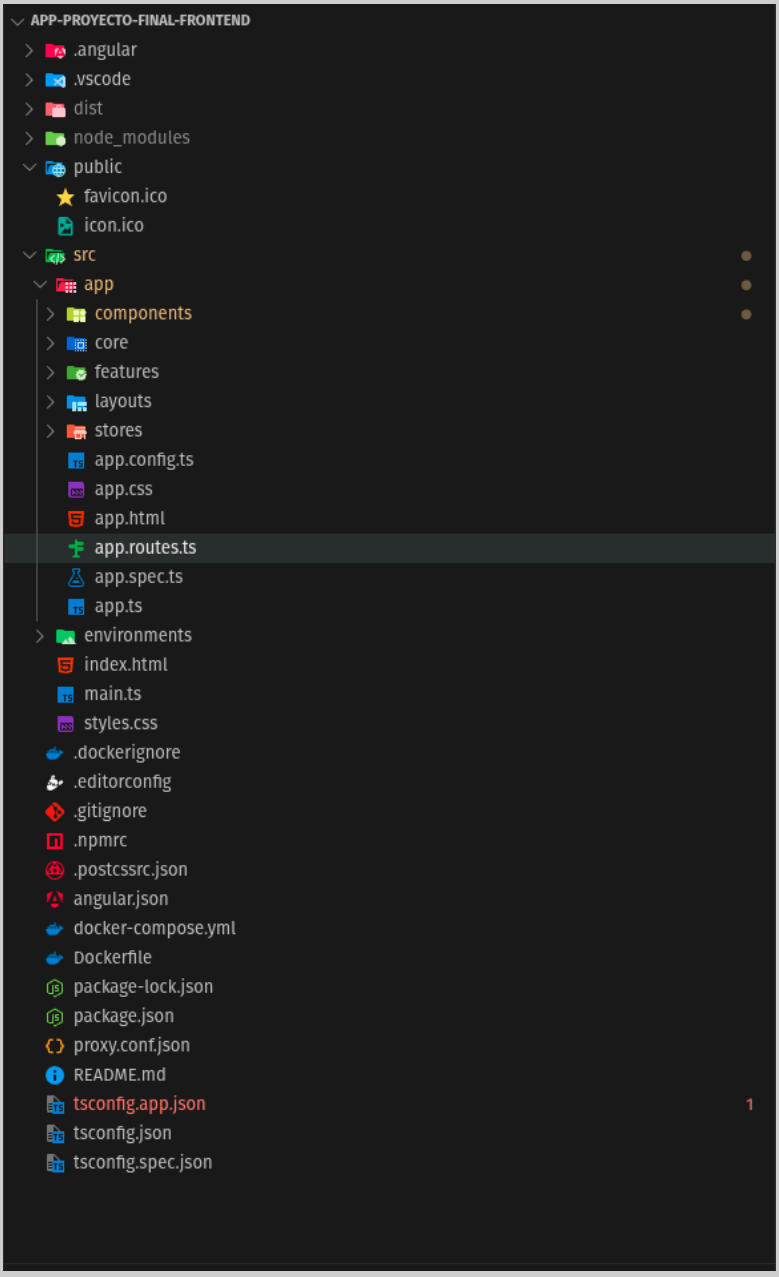
Es importante que el nuevo desarrollador mantenga las prácticas de fragmentación en componentes. Nunca generar clases enormes para tareas, siempre respetar los principios SOLID.

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro del paquete ubicado en la raíz llamado **documentación.**

DIRECTORIO DE ARCHIVOS APLICACIÓN FRONTEND

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de variables y carpetas.



Detalles importantes:

- **La carpeta “core”:** es donde se encuentra toda la lógica en común que tiene la aplicación y sobre todo que tiene la protección de URL.
- **La carpeta “components”:** es donde se aloja toda la capa de componentes consumidos por la aplicación para poder ensamblar las vistas
- **La carpeta: “features”:** Maneja toda la lógica de negocio perteneciente a todos los modelos que va a utilizar el frontend para poder manejar sus vistas.
- **La carpeta: “layouts”:** Maneja todos los presets de layouts y vistas que ofrece el sistema.

Este paquete es el que gestiona por completo las vistas de la ui usando componentes personalizados escritos en angular y sobreescritura de las propias librerías de node**. Es importante que el nuevo desarrollador mantenga las prácticas de fragmentación en componentes. Nunca generar componente enormes para tareas, siempre respetar los principios SOLID.**

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro del paquete ubicado en la raiz llamado **documentación.**

TECNOLOGÍAS UTILIZADAS

Arquitectura de software:

Arquitectura por capas

Patrones de diseño:

1. Patrón experto
2. Patrón Observer
3. Patrón Facade
4. Patrón Strategy
5. Patrón Factory

Paradigmas utilizados:

- Programación orientado a objetos
- Programación funcional

Principios SOLID

Otras tecnologías utilizadas:

- Modelo MVC

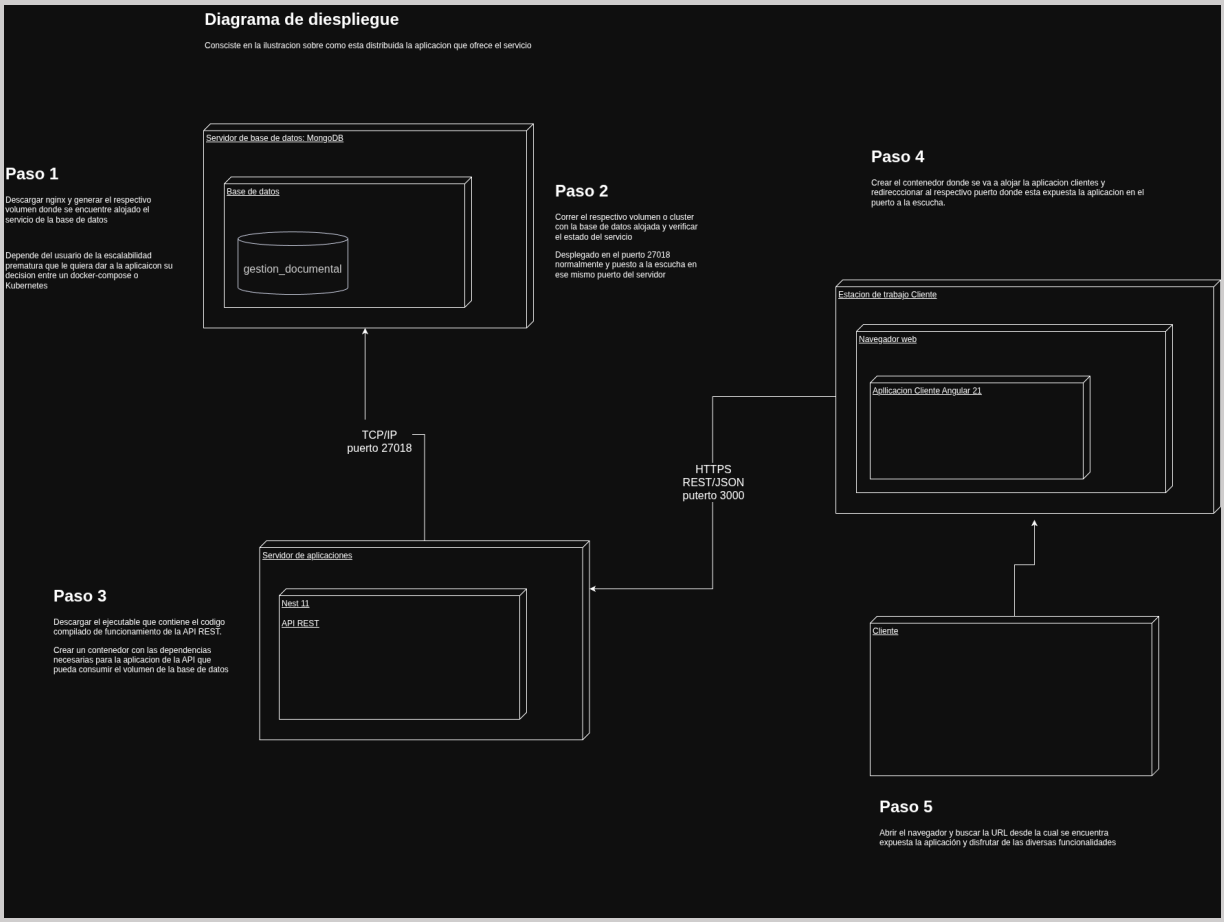
Debido a que la aplicación fue desarrollada en el entorno de NodeJS, se recomienda tener instalado:

- Editor de texto o IDE: ya sea Visual studio code o algun IDE especializado que soporte estas extensiones y tecnologías con la extensión de NodeJS
- Tener instalada la versión 22 de NodeJS
- **Tener instalada la extensión de Nest**
- **Tener instalada la extensión de Angular**

DIAGRAMA DE DESPLIEGUE

Este es uno de los diagramas que describe cómo es que está distribuida la aplicación. Y sobre todo ayuda a describir al desarrollador o encargado de la distribución de la aplicación como se puede lanzar a producción.

Aca se deja la implementación base:

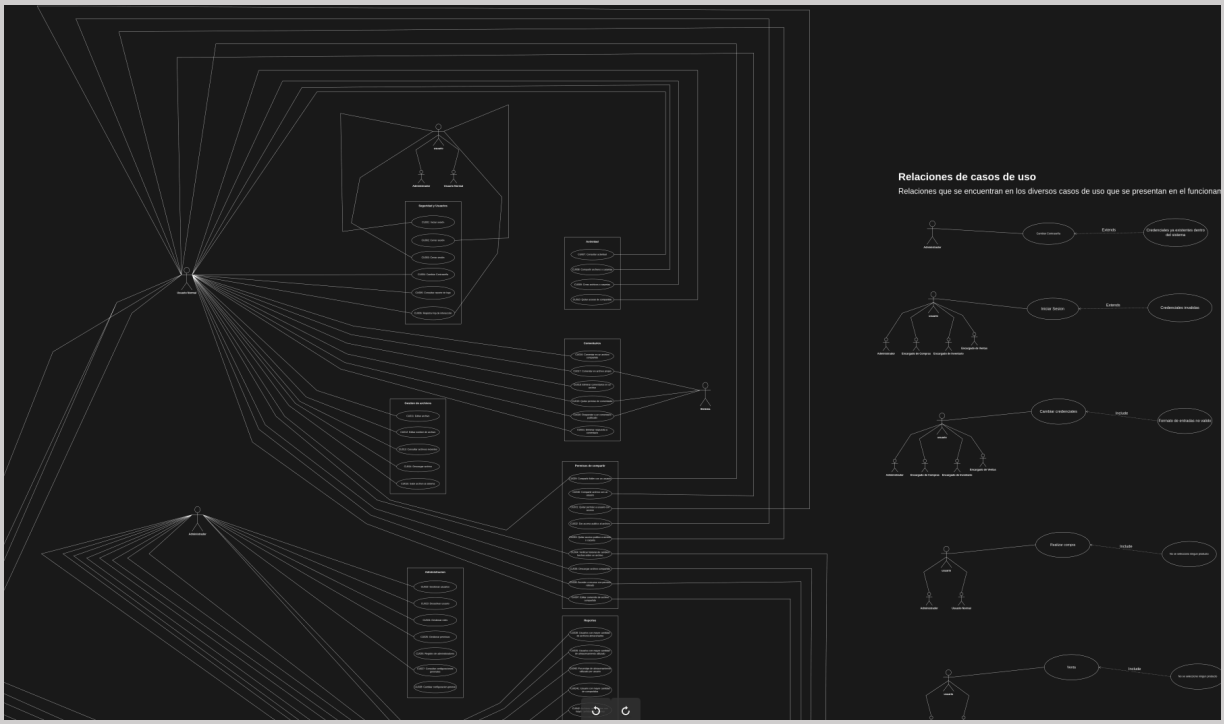


Puedes verlo más a detalle en el link:

https://drive.google.com/file/d/1NDkT9ZDRyl49Uiaju5-i4gujN00S69IL/view?usp=sharing

DIAGRAMA DE CASOS DE USO

Este es uno de los diagramas que describe cómo está organizada la aplicación y que casos pueden existir.

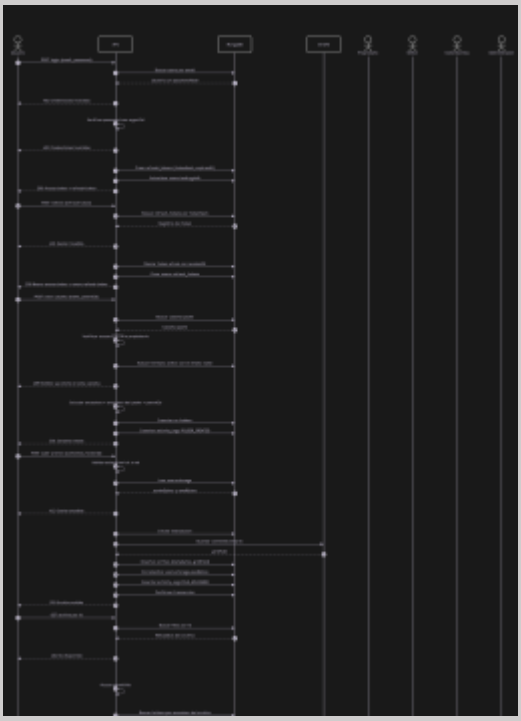


Puedes verlo más a detalle en el link:

https://drive.google.com/file/d/1NRFQdM25HnR5a1Ddv-smVzMQTpoDLp7T/view?usp=sharing

DIAGRAMA DE SECUENCIA

Este es uno de los diagramas que describe como es el flujo que sigue la aplicación al momento de ejecutar diferentes actividades o funciones.



Puedes verlo más a detalle en el link:

https://drive.google.com/file/d/1OyyKmvFfWBJ9IPXl2Sxsnp-l9a3V5STl/view?usp=sharing